

UP24 Lab05

Date: 2024-05-27

- [UP24 Lab05](#)
- [Buffer Overflow & Shellcoding Challenges](#)
 - [Challenge #1](#)
 - [Challenge #2](#)
 - [Challenge #3](#)
 - [Challenge #4](#)
- [Hints](#)
 - [How Many Bytes Are Required to Overflow a Stack?](#)
 - [Leak Stack Information](#)
 - [Return-Oriented Programming](#)
 - [Work with Apple Chip Computers](#)
 - [Sample Pwntool Script](#)
- [Lab Grading](#)
- [Lab Submission](#)

Buffer Overflow & Shellcoding Challenges

This lab aims to practice advanced assembly tricks. Specifically, we focus on implementing buffer overflow attacks and writing shellcodes. Please read the codes and solve the challenges available on the four challenge servers.

Note that the difficulty of the challenges could be diverse. I personally recommend solving the challenges in the order.

Challenge #1

The challenge server #1 can be accessed using the command:

```
nc up.zoolab.org 10257
```

The challenge server simply runs the shellcodes you offered. Please try to invoke a shell, e.g., `/bin/sh`, from your shellcode and then dump the FLAG in `/FLAG` in the shell.

We provide the source code and the binary of the challenge server for your reference [[source code](#) | [binary](#)].

Challenge #2

The challenge server #2 can be accessed using the command:

```
nc up.zoolab.org 10258
```

Once connected to the challenge server, please send your shellcode in the `msg` buffer, invoke your shell code (using buffer overflow), and dump the FLAG in `/FLAG`. You may need to leak the required information from the stack to achieve the goal.

We provide the source code and the binary of the challenge server for your reference [[source code](#) | [binary](#)].

The challenge has an obvious buffer overflow problem in its implementation. Also, note that there is ***no canary*** protection for the vulnerable function.

The reports from `checksec` may report having ***canaries*** in the executable, but don't worry about that.

Challenge #3

The challenge server #3 can be accessed using the command:

```
nc up.zoolab.org 10259
```

Once connected to the challenge server, please send your shellcode in the `msg` buffer, invoke your shell code (using buffer overflow), and dump the FLAG in `/FLAG`. You may need to leak the required information from the stack to achieve the goal.

We provide the source code and the binary of the challenge server for your reference [[source code](#) | [binary](#)].

The challenge has an obvious buffer overflow problem in its implementation. ***The source code of bof2 is exactly the same as bof1.*** The only difference is that we have ***canary protection enabled*** for the bof2 executable.

Challenge #4

The challenge server #4 can be accessed using the command:

```
nc up.zoolab.org 10261
```

Once you have connected to the challenge server, please try to send your shellcode and dump the FLAG in `/FLAG`. You may need to leak the required information from the stack to achieve the goal.

We provide the source code and the binary of the challenge server for your reference [[source code](#) | [binary](#)].

The primary difference between this challenge and the previous ones is that we do not have a global executable buffer (`msg`) for this challenge. Therefore, you have to think about how to execute the shellcodes by yourself.

You need to know what ***returned-oriented programming (ROP)*** is first and then solve the challenge. More information can be found in the hint for [returned-oriented programming](#).

Hints

How Many Bytes Are Required to Overflow a Stack?

Suppose you do not know how many bytes are required to overflow a stack. In that case, you can run the target program locally using `gdb`, set a **breakpoint** right before you fill the buffer (receiving user inputs), and see how far it is from the buffer address to your target address in the stack.

Leak Stack Information

All the challenges in this lab are compiled with the `-static-pie` option, which could lead to randomly assigned runtime addresses. Therefore, you may need to leak the required information on the stack to determine the actual addresses used to load the program.

In the following example, the return address (`0x007ffff7f2eae4`) of the function `task` in `bof1` can be leaked from the stack. Since we have a `printf` function that dumps the buffer in `bof1`, we can simply overflow the buffer by filling 40-byte `A`s. Once the `printf` function shows the buffer's content, we can leak the actual return address of the `task` function. The leaked return address helps recover the base address of the program, and it can be further used to calculate the required address in the program.

```
0x007ffff7ffe440|+0x0000: 0x007ffff7ffe668 → 0x007ffff7ffe98c → "... " ← $rsp
0x007ffff7ffe448|+0x0008: 0x007ffff7ffe470 → 0x0000000000000001
0x007ffff7ffe450|+0x0010: 0x0000000000000001
0x007ffff7ffe458|+0x0018: 0x007ffff7ffe658 → 0x007ffff7ffe94b → "... " ← $rbp
0x007ffff7ffe460|+0x0020: 0x007ffff7ffe470 → 0x0000000000000001 ← $rbp
0x007ffff7ffe468|+0x0028: 0x007ffff7f2cae4 → <main+160> mov eax, 0x0
0x007ffff7ffe470|+0x0030: 0x0000000000000001
0x007ffff7ffe478|+0x0038: 0x007ffff7f2ef1a → <__libc_start_call_main+106> mov edi,
```

Return-Oriented Programming

The basic idea of return-oriented programming (ROP) is to reuse pieces of executable machine instructions (often called ***gadgets***) in the memory to construct required shellcodes. This is useful when you can perform buffer overflow, but finding an executable memory area for writing shellcodes is infeasible.

You may have a look at the [page](#) for introducing ROP concepts. For example, suppose we plan to run the following instructions in our shellcodes:

```
mov rdi, 1
mov rax, 60
syscall # exit(1)
```

You have to find ***gadgets*** that can be used to replace the involved instructions. Each gadget should be in the form of

- `pop ... ; ret`,
- `syscall , or`
- `syscall ; ret`.

Suppose you have found the instructions in the following addresses:

```
0x4000025340 : pop rax , ret
0x400000918f : pop rdi , ret
0x4000022115 : syscall , ret
```

Once a `ret` instruction is to be invoked, and the stack is filled with the following content:

```
+-----+
| 0x400000918f <pop_rdi> | <-- rsp is here
+-----+
| 0 |
+-----+
| 0x4000025340 <pop_rax> |
+-----+
| 60 |
+-----+
| 0x4000022115 <syscall> |
+-----+
```

Running the gadgets filled in the stack can appropriately assign the required values to `rdi` and `rax` and then invoke the `syscall` instruction.

You can use the `R0Pgadget` command to find the gadget addresses in an executable. For example,

```
R0Pgadget --binary bof1
```

Please note that the reported addresses are offsets to the loaded program base address because the binary is compiled with PIE-enabled. You still need to know the actual program-loaded base address to obtain the required addresses when the program is running.

In addition to the `R0Pgadget` command, you may also consider the `R0P` class in the `pwntools` library. See [here](#) in the official document.

Work with Apple Chip Computers

All the challenges in this lab require an Intel x86_64 CPU. If you are working with an Apple Chip computer, e.g., M1, M2, and M3, you can solve the challenges using the `qemu` user-mode emulator.

All the challenge binaries are compiled with the `-static-pie` option, making them self-contained without additional dependencies. Therefore, you can simply run the command in a ***native (aarch64-based) Linux docker*** using the command

```
qemu-x86_64-static ./bof1
```

Replace `bof1` with other challenge binaries would work. However, the base addresses used for loading a binary in `qemu` might be much different from those used for a native CPU. You may need to calculate the base address by yourself. It looks like, by default `qemu` will always use a fixed base address of `0x400000000` for loading the text section.

We did not check whether the ***Rosetta2*** emulator works. Please try it at your own risk.

The local runtime is only used for testing and debugging your solution. Our grading will only be based on the result of solving the challenge on the remote.

You may need to install the `gcc-multilib-x86-64-linux-gnu` package in your aarch64 Debian Linux distribution.

QEMU user mode does not support the `seccomp` feature. Therefore, you cannot use it to invoke the `shellcode` executable in challenge #1. If you need to run it locally on an apple chip computer, please set the `N0_SANDBOX` environment variable before running it to disable using `seccomp`. For example, run it using the command:

```
N0_SANDBOX=1 qemu-x86_64-static ./shellcode
```

Sample Pwntool Script

Here, we provide a sample script for you to run the challenge binaries locally or connect to the remote host. Replace `bof1` or the port number with the one you prefer. The script is available here [[view](#) | [download](#)].

Lab Grading

- [20 pts] You can solve challenge #1.
- [20 pts] You can solve challenge #2.
- [20 pts] You can solve challenge #3.
- [40 pts] You can solve challenge #4.

Lab Submission

You have to upload all your solution scripts and codes to e3.

- Filename: {studentID}_lab05.zip
- Format:

```
+---{studentID}_lab05
|  chal_1.py
|  chal_2.py
|  other files...
```

You must put your files in a directory first, then compress the directory.